R1.04 - Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Gestion des processus, cycle de vie, ordonnancement multitâche préemptif, filiation PID/PPID et communication inter-processus par signaux POSIX sous Unix/Linux

Synthèse exécutive : Ce cours aborde l'architecture des systèmes d'exploitation de type Unix/Linux à travers le concept fondamental de processus. Il explique la transition d'un programme statique stocké sur le disque dur vers un processus dynamique résidant en mémoire RAM. Pour faire fonctionner de nombreuses applications simultanément sur une machine possédant un nombre limité de cœurs processeurs, le noyau s'appuie sur le multitâche préemptif, gérant le cycle de vie des processus via trois états fondamentaux (Élu, Prêt, Bloqué). Le cours détaille l'arborescence généalogique parent-enfant (PID/PPID) issue du processus initial systemd/init (PID 1), le mécanisme de création par fork() et exec(), ainsi que le contrôle des tâches et la communication asynchrone au moyen des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Dans un système informatique moderne, le processeur doit gérer plusieurs applications simultanément. Pour y parvenir, le système d'exploitation fait basculer le code d'un état statique sur le disque dur vers un état actif en mémoire vive (RAM). Ce passage crée une entité vivante appelée processus. Le noyau Linux orchestre cette simultanéité via le multitâche préemptif. L'ordonnanceur (scheduler) du noyau ne laisse pas un processus occuper le processeur indéfiniment : il découpe le temps processeur en minuscules tranches (quanta de temps) qu'il distribue à tour de rôle. Le cycle de vie d'un processus s'articule autour de trois états majeurs : 1. Élu (Running) : Le processus occupe physiquement un cœur du processeur et ses instructions machine sont exécutées. 2. Prêt (Ready) : Le processus possède l'intégralité de sa structure en mémoire RAM et attend simplement que l'ordonnanceur lui attribue le prochain quantum de temps processeur. 3. Bloqué (Waiting / Blocked) : Le processus déclenche une requête nécessitant une attente externe (par exemple une lecture de fichier sur le disque dur, la réception d'un paquet réseau ou l'attente d'une saisie utilisateur au clavier). Le noyau libère immédiatement le processeur pour un autre processus et place la tâche en attente d'événement. Dès que l'événement survient, le processus ne retourne JAMAIS directement à l'état 'Élu' : il bascule impérativement dans l'état 'Prêt' pour être réinscrit dans la file d'attente de l'ordonnanceur.

- Programme vs Processus : Un programme est une suite d'instructions statique et passive stockée sur le disque dur. Un processus est l'instance active en cours d'exécution de ce programme chargée en mémoire RAM, consommant du temps CPU et disposant de ressources allouées.
- Multitâche Préemptif & Ordonnanceur : Mécanisme par lequel le noyau (kernel) découpe le temps processeur en minuscules quanta de temps et les alloue dynamiquement aux différents processus pour simuler une exécution simultanée.
- Process Control Block (PCB) : Structure de données interne du noyau conservant l'état du processeur, la carte mémoire (stack, heap, code, data), la priorité, les identifiants (PID, PPID, UID) et les descripteurs de fichiers ouverts pour chaque processus.
- Modèle des 3 États : Élu (Running - exécute son code sur le CPU), Prêt (Ready - dispose de ses ressources RAM et attend qu'un quantum CPU lui soit attribué), Bloqué (Waiting/Blocked - suspendu en attente d'une opération d'Entrée/Sortie ou d'un événement externe). Un processus Bloqué repasse toujours par l'état Prêt avant de repasser Élu.

Cas concret enseignant : *L'analogie du livre de recettes de cuisine : Le programme stocké sur le disque dur est comparable à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, occupant de la place en mémoire RAM sur le plan de travail, consommant du temps processeur CPU pour la cuisson et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

2. Filiation, Arborescence et Mécanismes de Création (fork et exec)

Au sein des systèmes Unix/Linux, aucun processus ne peut apparaître de manière spontanée. Tous les processus utilisateurs s'inscrivent dans une généalogie stricte. Lors de l'initialisation de l'ordinateur, le noyau démarre le tout premier processus utilisateur, nommé systemd (ou init sur d'anciennes distributions), qui reçoit de façon garantie le PID 1. Lorsqu'un utilisateur lance une commande dans un terminal (comme 'ls' ou 'vim'), le shell interactif (qui est lui-même un processus parent) utilise le mécanisme classique de création de tâches Unix en deux étapes : 1. fork() : Le processus père exécute l'appel système fork(). Le noyau duplique alors le père à l'identique. Un nouvel identifiant (PID) est assigné au fils, et le PPID du fils prend la valeur du PID du père. 2. exec() : Immédiatement après la bifurcation, le processus fils appelle l'une des fonctions de la famille exec(). Le noyau remplace l'intégralité du code et des données hérites de l'enfant par les nouvelles instructions binaire de l'application visée (ex: /usr/bin/ls). Le processus conserve son PID mais exécute désormais le nouveau code.

- Arborescence et Identifiants (PID / PPID) : Chaque processus possède un Process Identifier (PID) unique attribué par le noyau et pointe vers le Parent Process ID (PPID) du processus qui l'a créé. Tous les processus forment un arbre dont la racine unique est le processus initial systemd ou init (PID 1).
- Appel système fork() : Fonction du noyau qui duplique le processus parent à l'identique pour créer un processus enfant possédant son propre PID et une copie exacte du contexte mémoire.
- Appel système exec() : Fonction du noyau qui reemplace l'image mémoire du processus appelant par le code binaire d'un nouveau programme à exécuter.

3. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent la forme de communication inter-processus (IPC) la plus élémentaire sous Unix. Lorsqu'un signal est adressé à un processus cible, le noyau interrompt immédiatement le fil séquentiel des instructions pour exécuter l'action requise. Face à un signal, trois comportements sont possibles pour le processus : 1. Exécuter l'action par défaut du système (souvent la terminaison de la tâche). 2. Ignorer le signal (ignore). 3. Intercepter le signal (catch) en associant une fonction personnalisée (le gestionnaire de signal). Cela permet par exemple à un traitement de texte d'intercepter SIGTERM pour afficher 'Voulez-vous enregistrer votre document ?' avant d'éteindre l'application. Toutefois, afin de préserver l'autorité absolue de l'administrateur système et permettre de neutraliser un programme défaillant, la norme POSIX définit deux exceptions incapturables : SIGKILL (Signal 9) et SIGSTOP (Signal 19). Ces deux signaux sont traités en direct par le noyau lui-même. Aucun code applicatif ne peut les détourner ou les ignorer.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour lui notifier un événement ou lui ordonner une instruction.
- Gestionnaire de signal (Signal Handler) : Routine logicielle enregistrée par le développeur pour intercepter (catch) un signal et en exécuter un traitement sur-mesure au lieu du comportement système par défaut.
- SIGINT (Signal 2) : Déclenché par 'Ctrl + C' dans le terminal. Demande une interruption douce et modérée d'une tâche interactive.
- SIGTERM (Signal 15) : Signal d'arrêt propre envoyé par défaut par la commande 'kill '. Il invite le processus à libérer ses ressources, fermer ses connexions et sauvegarder ses fichiers avant de s'arrêter.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel et immédiat exécuté directement par le noyau Linux. STRICTEMENT IMPOSSIBLE à intercepter (catch), masquer ou ignorer.
- Job Control (SIGSTOP 19 / SIGCONT 18) : 'Ctrl + Z' émet SIGSTOP pour suspendre l'exécution d'une tâche ; les commandes 'fg' ou 'bg' émettent SIGCONT pour reprendre son exécution.

Cas concret enseignant : *L'analogie de l'expulsion du bâtiment : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous disposez du temps nécessaire pour sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (Signal 9) représente l'intervention de la brigade de sécurité d'urgence qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : La commande 'kill 9 1234' interprète le chiffre '9' et le chiffre '1234' comme étant DEUX identifiants de processus (PID) distincts auxquels elle essaie d'envoyer le signal par défaut (SIGTERM 15). Elle échoue car le PID 9 n'existe généralement pas. En revanche, le tiret '-' dans 'kill -9 1234' passe le chiffre 9 comme une option spécifiant le numéro du signal à envoyer (SIGKILL), suivi du PID cible 1234.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' dans le système et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie (état 'Z' dans la commande ps) est un processus enfant qui a terminé son exécution, mais dont le père n'a pas encore lu le code de retour de fin d'exécution via l'appel système wait(). Le zombie ne consomme plus aucune mémoire RAM ni aucun cycle CPU : il ne reste de lui qu'une ligne dans la table des processus (un PCB minimal). Il est donc IMPOSSIBLE de le tuer avec 'kill' (y compris 'kill -9') car le processus est DÉJÀ mort. Pour le faire disparaître, il faut soit forcer le père à exécuter wait(), soit tuer le processus père (PPID). En mourant, le père orpheline le zombie, qui est alors immédiatement adopté par le processus racine systemd/init (PID 1), lequel effectue automatiquement le nettoyage du PCB.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&' à la fin de la première ligne, détaillez le comportement produit par l'envoi du signal -15 et décrivez le résultat obtenu lors d'un contrôle ultérieur par la commande ps.

Démarche de résolution :
Étape 1 : Le symbole '&' placé à la fin de la commande 'sleep 600 &' demande au shell interactif d'exécuter la tâche en arrière-plan (background). Le shell affiche instantanément le job ID (ex: [1]) et le PID attribué par le noyau (ex: 4521), puis redonne immédiatement la main à l'utilisateur sans bloquer l'invite de commande. Étape 2 : La commande 'ps -ef | grep sleep' filtre la liste globale des processus système. La sortie confirme que le processus 'sleep 600' est actif avec le PID 4521 et identifie son PPID (le PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus. L'utilitaire sleep ne possédant pas de gestionnaire spécifique pour capturer SIGTERM, l'action par défaut du système s'applique : l'arrêt propre et la suppression du processus.

```
#!/bin/bash
# Séquence de démonstration de contrôle des tâches sous Linux

# 1. Lancement de la commande sleep en arrière-plan
sleep 600 &

# 2. Identification du PID du processus sleep
ps -ef | grep sleep

# 3. Terminaison propre du processus avec SIGTERM (Signal 15)
kill -15 4521
```

Résultat : Le processus 'sleep 600' est détruit de manière propre par le signal SIGTERM. Une vérification ultérieure au moyen de 'ps -ef | grep sleep' confirme sa disparition complète de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement du système ?

R : Le tout premier processus utilisateur est 'systemd' (ou 'init' sur les anciennes distributions) et son PID est obligatoirement 1.

Q : Quel signal POSIX est généré et transmis au processus en premier plan lorsque l'utilisateur effectue la combinaison de touches 'Ctrl + C' dans son terminal ?

R : La combinaison 'Ctrl + C' génère le signal SIGINT (Signal n°2), qui demande l'interruption douce de l'application interactive.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : Parce que la norme POSIX impose que SIGKILL soit intercepté et exécuté directement par le noyau lui-même. Le signal n'est jamais transmis au code applicatif du processus, garantissant à l'administrateur le pouvoir absolu de stopper n'importe quelle tâche.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765